FIRST EDITION



PROGRAMMING WITH FUBBY

PROGRAMMING LANGUAGE

A Practical Guide to the Fubby Programming Language

TUSHAR H. KSHIRSAGAR

Twig Software Solutions Pvt. Ltd.

PROGRAMMING WITH FUBBY

A Practical Guide to the Fubby Programming Language

First Edition | Student and Instructor Edition

Tushar H. Kshirsagar

Twig Software Solutions Pvt. Ltd.

Developer Preview v0.1.0 • November 2026

Copyright and publication information

Copyright © 2026 Tushar H. Kshirsagar. All rights reserved.
Publisher/imprint: Twig Software Solutions Pvt. Ltd. (subject to publisher confirmation).

ISBN: Not yet assigned. An ISBN must be obtained from an authorized registration agency before publication; no number has been invented for this edition.

Edition: First edition, manuscript/print-proof version, November 2026. Format: 6 × 9 in paperback interior. Language: English.

Fubby is proprietary software. The public examples are instructional source code, not the Rust interpreter source. The terms governing redistribution, use of the book, and licensing should be reviewed by the rights holder before commercial printing or sale.

Technical notice: Fubby v0.1.0 is an experimental developer preview. Features, command-line behavior, packaging, and APIs may change. Do not execute untrusted .fab programs. Web and database demonstrations are for local development and do not constitute a security-hardened production service.

Source references: github.com/tusharuit25/fubby-releases and its examples/ directory; this edition was prepared from the supplied README.md and examples.zip snapshot. Code excerpts were not re-executed during this typesetting pass.

Printed in: _____ ISBN: _____

Dedication

To curious students who learn a language by making something work, changing it, breaking it, and understanding why.

Foreword

A new programming language offers students an unusual opportunity: they can learn not only how to write programs, but how a language is organized into its syntax, runtime, libraries, tools, and applications. Fubby is an experimental system, so the discipline of observing behavior and checking assumptions matters as much as learning individual commands.

This academic edition begins with the smallest runnable programs and gradually introduces reusable abstractions, collections, packages, web services, and concurrency. Each topic is anchored to the publicly distributed examples rather than invented syntax. Students are encouraged to read source, run local experiments, document observations, and distinguish tested facts from hypotheses.

The foreword is an editorial introduction, not an endorsement or testimonial by an outside contributor. A signed external foreword may be commissioned for a later published edition.

Acknowledgements

This manuscript draws on the Fubby developer-preview documentation and the fifteen example projects provided for student learning. Credit is due to the developers and reviewers who made repeatable examples and verification procedures a priority. Individual contributor names should be added only after they have been confirmed and permission to credit them has been obtained.

Special acknowledgement is given to students and educators who approach new technology with curiosity and careful skepticism. Their questions about error messages, edge cases, portability, and security are essential to improving any teaching resource.

Preface

Programming is best learned through small, complete experiments. Fubby provides a compact environment in which students can explore expressions, control flow, functions, classes, collections, packages, web applications, and concurrency. This book is a guided companion to the publicly distributed example projects.

The fifteen lessons progress from a first greeting to an integrated application. Each chapter states what to learn, how to run the official example, what to observe, and how to extend it. Students should type and modify code, not merely read it.

The material is written for beginners who can use a terminal. Instructors may use the review questions, checkpoints, and final project as classroom exercises. Some advanced chapters are optional for an introductory course.

A note on versions: The supplied top-level README describes a preview. Its release asset names and extraction commands differ from the verified M20 archive identified by the examples. Always inspect the archive layout and follow the installer bundled with the actual distribution; do not assume the older README extraction path applies to M20.

Tushar H. Kshirsagar

How to use this book

• Read the learning goals before opening the chapter project.

• Locate the chapter folder inside examples/ and read its README.md.

• Run the documented command with the same Fubby build used by the example author.

• Compare actual behavior with the documented expected behavior.

• Complete the three activities and record what changed.

• If a command fails, check the path, interpreter version, dependencies, and terminal output before changing code.

Suggested pacing: Lessons 1–6 in weeks 1–4; lessons 7–11 in weeks 5–7; lessons 12–15 and the capstone in weeks 8–10. Adjust for students' prior programming experience.

For code and downloads: github.com/tusharuit25/fubby-releases

Contents

A descriptive guide to the chapters and reference sections

01 **Hello** 11
First program, variables, constants, arithmetic

02 **Control Flow** 19
Conditions, branching, loops, and program decisions

03 **Functions** 29
Reusable logic, parameters, and return values

04 **Classes** 39
Objects, state, methods, and encapsulation

05 **Inheritance** 49
Extending classes and reusing behavior

06 **Collections, Imports & Errors** 60
Lists, maps, modules, and exception handling

07 **Standard Library** 68
Built-in utilities and practical operations

08 **Runtime** 78
Running scripts, CLI workflows, and execution behavior

09 **Core Library** 87
Working with Fubby Core capabilities

10 **Moon** 98
HTTP requests, local services, and SQLite

11 **Shady** 108
Testing, formatting, linting, and debugging

12 **Snow** 118
Web pages, APIs, serving, and shutdown

13 **Bela** 129
Concurrency concepts and coordinated tasks

14 **Smokey** 139
Packages, installation, and validation

15 **Integrated Application** 146
Combining libraries in a complete project

Capstone project Final applied assessment 154

Glossary Definitions of key Fubby terms 156

Troubleshooting Common errors and fixes 158

Subject index Alphabetical topics with page references 160

Getting started: installation and safety

The verified M20 Intel macOS archive is fubby-0.1.0-x86_64-apple-darwin.tar.gz. Its SHA-256 checksum is:

```
505c61cfa826764a4eb127acff5cacb2d8b9d67805bd0210bb965764bb52a670
```

The archive was reported as verified on Intel macOS. Apple Silicon, Linux, and Windows should not be assumed supported by this particular build. On macOS, inspect the extracted package and use its bundled installation instructions.

```
shasum -a 256 fubby-0.1.0-x86_64-apple-darwin.tar.gz
tar -tzf fubby-0.1.0-x86_64-apple-darwin.tar.gz | head
fubby --version
fubby --help
```

Useful CLI operations documented by the supplied README:

```
fubby run hello.fab
fubby check hello.fab
fubby init my-project
fubby repl
```

The v0.1 REPL is stateless across entered lines. Never assume that running an untrusted .fab file is sandboxed. Keep HTTP and SQLite practice data local and disposable.

Chapter 01 Hello

Official project: examples/01-hello/

Milestones: M1. Print a greeting, declare variables and constants, and evaluate arithmetic. With Fubby 0.1.0 on PATH, open a terminal in this folder:

Learning objectives

• Create and run a .fab file.

• Recognize a main function and print statements.

• Change an expression and predict the result.

Run the official example

```
fubby check hello.fab
fubby run hello.fab
fubby check basics.fab
fubby run basics.fab
```

Read the program

Excerpt from: 01-hello/hello.fab

```
func main() {
    print("Hello, Fubby!")
}
```

What to notice

A program is a sequence of instructions. Read the entry point and compare printed values with the source.

Try it yourself

Activity 1. Run the unmodified 01-hello example and record the actual output or observable behavior.

Activity 2. Change one input, literal, or configuration value. Predict the result before rerunning.

Activity 3. Explain one error condition or edge case and how you would test it.

Check your understanding

Question 1. Which line or file provides the entry point for this project?

Question 2. Which concept from this chapter would be most useful in a larger application, and why?

Question 3. What evidence would show that your modified example still works?

Instructor note. Assess the learner’s prediction, observed output, and explanation, not merely successful execution.

Lab A | Source-map investigation

Before editing a program, map the relationship between its entry point, local modules, assets, and README. A source map is a human-readable account of which file has which responsibility.

Source reading: 01-hello/basics.fab

```
func main() {
    let name = "Fubby"
    const version = 1
    let answer = 6 * 7
    print("Language:", name)
    print("Version:", version)
    print("6 * 7 =", answer)
}
```

1. List the project files and identify which ones contain executable Fubby statements.

2. Find the documented working directory and explain why running from a different directory might fail.

3. Draw a dependency arrow from the entry file to each imported file, package, or external service.

4. State one feature of the example that is explicitly documented and one detail you still need to verify.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab B | Reading program behavior

Practice a dry run: trace control flow without executing the program. For a web or concurrent project, trace a single request or task instead of assuming a globally fixed execution order.

Source reading: 01-hello/hello.fab

```
func main() {
    print("Hello, Fubby!")
}
```

1. Identify the first observable operation and the values it depends on.

2. Write a step-by-step trace for a representative successful path.

3. Identify a branch, function call, or external boundary that could change the trace.

4. Run the official example and compare your trace with actual output or behavior.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab D | Edge cases and failure paths

A program that succeeds on its demonstration input is not automatically robust. Deliberately examine missing values, invalid inputs, empty collections, unavailable services, and incompatible environments.

Source reading: 01-hello/README.md

````
# Hello and expressions

Milestones: M1.

Print a greeting, declare variables and constants, and evaluate
arit ...

## Run

With Fubby 0.1.0 on PATH, open a terminal in this folder:

```sh
fubby check hello.fab
fubby run hello.fab
fubby check basics.fab
fubby run basics.fab
```

## What to expect

Hello, Fubby!; the basics example prints 42.

## Try it

Change an input value in the example, rerun it, and compare the
resu ...

[All examples](../README.md)
````

1. List three edge cases relevant to this chapter, ordered by likelihood.

2. Predict the behavior for one invalid input before running it.

3. Where possible, reproduce a safe failure using disposable local data.

4. Explain whether the failure is handled by Fubby code, the interpreter, or the operating environment.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab F | Verification worksheet

Reproducibility requires more than saying that a program works. A useful verification record identifies the exact command, environment, expected behavior, and observed result.

Source reading: 01-hello/README.md

````
# Hello and expressions

Milestones: M1.

Print a greeting, declare variables and constants, and evaluate
arit ...

## Run

With Fubby 0.1.0 on PATH, open a terminal in this folder:

```sh
fubby check hello.fab
fubby run hello.fab
fubby check basics.fab
fubby run basics.fab
```

## What to expect

Hello, Fubby!; the basics example prints 42.

## Try it

Change an input value in the example, rerun it, and compare the
resu ...

[All examples](../README.md)
````

1. Write the exact command from the project README, including its working directory.

2. Record the interpreter version and whether external services or packages were needed.

3. Define one positive test and one negative test with explicit expected observations.

4. If Shady or another tool applies, record its output and distinguish warnings from failures.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Chapter 02 Control Flow

Official project: examples/02-control-flow/

Milestones: M2. Use typed values, conditions, while loops, and ranges. With Fubby 0.1.0 on PATH, open a terminal in this folder:

Learning objectives

• Explain variables and basic types.

• Trace if, while, and for control flow.

• Recognize loop boundaries.

Run the official example

```
fubby check milestone2.fab
fubby run milestone2.fab
```

Read the program

Excerpt from: 02-control-flow/milestone2.fab

```
func main() {
    let String language = "Fubby"
    let Int score = 72
    let Float version = 0.1
    let Bool ready = true

    print(language, "version", version)

    if ready && score >= 60 {
        print("Milestone 2 is alive")
    } else {
        print("Not ready")
    }

    let i = 0
    while i < 3 {
        print("while", i)
        i += 1
```

```
    }

    for n in 1..5 {
        print("range", n)
    }
}
```

What to notice

Follow the value of each variable after every iteration. Check whether a range includes its final number.

Try it yourself

Activity 1. Run the unmodified 02-control-flow example and record the actual output or observable behavior.

Activity 2. Change one input, literal, or configuration value. Predict the result before rerunning.

Activity 3. Explain one error condition or edge case and how you would test it.

Check your understanding

Question 1. Which line or file provides the entry point for this project?

Question 2. Which concept from this chapter would be most useful in a larger application, and why?

Question 3. What evidence would show that your modified example still works?

Instructor note. Assess the learner’s prediction, observed output, and explanation, not merely successful execution.

Lab A | Source-map investigation

Before editing a program, map the relationship between its entry point, local modules, assets, and README. A source map is a human-readable account of which file has which responsibility.

Source reading: 02-control-flow/milestone2.fab

```
func main() {
    let String language = "Fubby"
    let Int score = 72
    let Float version = 0.1
    let Bool ready = true

    print(language, "version", version)

    if ready && score >= 60 {
        print("Milestone 2 is alive")
    } else {
        print("Not ready")
    }

    let i = 0
    while i < 3 {
        print("while", i)
        i += 1
    }

    for n in 1..5 {
        print("range", n)
    }
}
```

1. List the project files and identify which ones contain executable Fubby statements.

2. Find the documented working directory and explain why running from a different directory might fail.

3. Draw a dependency arrow from the entry file to each imported file, package, or external service.

4. State one feature of the example that is explicitly documented and one detail you still need to verify.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab B | Reading program behavior

Practice a dry run: trace control flow without executing the program. For a web or concurrent project, trace a single request or task instead of assuming a globally fixed execution order.

Source reading: 02-control-flow/README.md

````
# Types and control flow

Milestones: M2.

Use typed values, conditions, while loops, and ranges.

## Run

With Fubby 0.1.0 on PATH, open a terminal in this folder:

```sh
fubby check milestone2.fab
fubby run milestone2.fab
```

## What to expect

A readiness message, while values 0–2, and range values 1–5.

## Try it

Change an input value in the example, rerun it, and compare the
resu ...

[All examples](../README.md)
````

1. Identify the first observable operation and the values it depends on.

2. Write a step-by-step trace for a representative successful path.

3. Identify a branch, function call, or external boundary that could change the trace.

4. Run the official example and compare your trace with actual output or behavior.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab D | Edge cases and failure paths

A program that succeeds on its demonstration input is not automatically robust. Deliberately examine missing values, invalid inputs, empty collections, unavailable services, and incompatible environments.

Source reading: 02-control-flow/README.md

````
# Types and control flow

Milestones: M2.

Use typed values, conditions, while loops, and ranges.

## Run

With Fubby 0.1.0 on PATH, open a terminal in this folder:

```sh
fubby check milestone2.fab
fubby run milestone2.fab
```

## What to expect

A readiness message, while values 0–2, and range values 1–5.

## Try it

Change an input value in the example, rerun it, and compare the
resu ...

[All examples](../README.md)
````

1. List three edge cases relevant to this chapter, ordered by likelihood.

2. Predict the behavior for one invalid input before running it.

3. Where possible, reproduce a safe failure using disposable local data.

4. Explain whether the failure is handled by Fubby code, the interpreter, or the operating environment.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab F | Verification worksheet

Reproducibility requires more than saying that a program works. A useful verification record identifies the exact command, environment, expected behavior, and observed result.

Source reading: 02-control-flow/README.md

````
# Types and control flow

Milestones: M2.

Use typed values, conditions, while loops, and ranges.

## Run

With Fubby 0.1.0 on PATH, open a terminal in this folder:

```sh
fubby check milestone2.fab
fubby run milestone2.fab
```

## What to expect

A readiness message, while values 0–2, and range values 1–5.

## Try it

Change an input value in the example, rerun it, and compare the
resu ...

[All examples](../README.md)
````

1. Write the exact command from the project README, including its working directory.

2. Record the interpreter version and whether external services or packages were needed.

3. Define one positive test and one negative test with explicit expected observations.

4. If Shady or another tool applies, record its output and distinguish warnings from failures.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Chapter 03 Functions

Official project: examples/03-functions/

Milestones: M3. Pass arguments, return typed results, and calculate a factorial recursively. With Fubby 0.1.0 on PATH, open a terminal in this folder:

Learning objectives

• Define functions with parameters and returns.

• Trace a recursive call.

• Identify a termination condition.

Run the official example

```
fubby check milestone3.fab
fubby run milestone3.fab
```

Read the program

Excerpt from: 03-functions/milestone3.fab

```
func add(Int a, Int b) -> Int {
    return a + b
}
func factorial(Int n) -> Int {
    if n <= 1 { return 1 }
    return n * factorial(n - 1)
}
func greet(String name) { print("Hello", name) }
func main() {
    greet("Fubby")
    let Int total = add(20, 22)
    print("20 + 22 =", total)
    print("factorial 5 =", factorial(5))
}
```

What to notice

A function can call another function or itself. Recursive solutions need a base case.

Try it yourself

Activity 1. Run the unmodified 03-functions example and record the actual output or observable behavior.

Activity 2. Change one input, literal, or configuration value. Predict the result before rerunning.

Activity 3. Explain one error condition or edge case and how you would test it.

Check your understanding

Question 1. Which line or file provides the entry point for this project?

Question 2. Which concept from this chapter would be most useful in a larger application, and why?

Question 3. What evidence would show that your modified example still works?

Instructor note. Assess the learner’s prediction, observed output, and explanation, not merely successful execution.

Lab A | Source-map investigation

Before editing a program, map the relationship between its entry point, local modules, assets, and README. A source map is a human-readable account of which file has which responsibility.

Source reading: 03-functions/milestone3.fab

```
func add(Int a, Int b) -> Int {
    return a + b
}
func factorial(Int n) -> Int {
    if n <= 1 { return 1 }
    return n * factorial(n - 1)
}
func greet(String name) { print("Hello", name) }
func main() {
    greet("Fubby")
    let Int total = add(20, 22)
    print("20 + 22 =", total)
    print("factorial 5 =", factorial(5))
}
```

1. List the project files and identify which ones contain executable Fubby statements.

2. Find the documented working directory and explain why running from a different directory might fail.

3. Draw a dependency arrow from the entry file to each imported file, package, or external service.

4. State one feature of the example that is explicitly documented and one detail you still need to verify.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

Lab B | Reading program behavior

Practice a dry run: trace control flow without executing the program. For a web or concurrent project, trace a single request or task instead of assuming a globally fixed execution order.

Source reading: 03-functions/README.md

````
# Functions and recursion

Milestones: M3.

Pass arguments, return typed results, and calculate a factorial
recu ...

## Run

With Fubby 0.1.0 on PATH, open a terminal in this folder:

```sh
fubby check milestone3.fab
fubby run milestone3.fab
```

## What to expect

20 + 22 = 42 and factorial 5 = 120.

## Try it

Change an input value in the example, rerun it, and compare the
resu ...

[All examples](../README.md)
````

1. Identify the first observable operation and the values it depends on.

2. Write a step-by-step trace for a representative successful path.

3. Identify a branch, function call, or external boundary that could change the trace.

4. Run the official example and compare your trace with actual output or behavior.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab D | Edge cases and failure paths

A program that succeeds on its demonstration input is not automatically robust. Deliberately examine missing values, invalid inputs, empty collections, unavailable services, and incompatible environments.

Source reading: 03-functions/README.md

````
# Functions and recursion

Milestones: M3.

Pass arguments, return typed results, and calculate a factorial
recu ...

## Run

With Fubby 0.1.0 on PATH, open a terminal in this folder:

```sh
fubby check milestone3.fab
fubby run milestone3.fab
```

## What to expect

20 + 22 = 42 and factorial 5 = 120.

## Try it

Change an input value in the example, rerun it, and compare the
resu ...

[All examples](../README.md)
````

1. List three edge cases relevant to this chapter, ordered by likelihood.

2. Predict the behavior for one invalid input before running it.

3. Where possible, reproduce a safe failure using disposable local data.

4. Explain whether the failure is handled by Fubby code, the interpreter, or the operating environment.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab F | Verification worksheet

Reproducibility requires more than saying that a program works. A useful verification record identifies the exact command, environment, expected behavior, and observed result.

Source reading: 03-functions/README.md

````
# Functions and recursion

Milestones: M3.

Pass arguments, return typed results, and calculate a factorial
recu ...

## Run

With Fubby 0.1.0 on PATH, open a terminal in this folder:

```sh
fubby check milestone3.fab
fubby run milestone3.fab
```

## What to expect

20 + 22 = 42 and factorial 5 = 120.

## Try it

Change an input value in the example, rerun it, and compare the
resu ...

[All examples](../README.md)
````

1. Write the exact command from the project README, including its working directory.

2. Record the interpreter version and whether external services or packages were needed.

3. Define one positive test and one negative test with explicit expected observations.

4. If Shady or another tool applies, record its output and distinguish warnings from failures.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Chapter 04 Classes

Official project: examples/04-classes/

Milestones: M4. Construct an account and modify private state through public methods. With Fubby 0.1.0 on PATH, open a terminal in this folder:

Learning objectives

• Define classes and create objects.

• Read member access and encapsulation.

• Separate object state from behavior.

Run the official example

```
fubby check milestone4.fab
fubby run milestone4.fab
```

Read the program

Excerpt from: 04-classes/milestone4.fab

```
class BankAccount {
    private String owner
    private Float balance

    init(String owner, Float openingBalance) {
        this.owner = owner
        this.balance = openingBalance
    }

    public func deposit(Float amount) {
        this.balance += amount
    }

    public func getBalance() -> Float {
        return this.balance
    }
}
```

```
func main() {
    let account = BankAccount("Tushar", 5000.0)
    account.deposit(2500.0)
    print(account.getBalance())
}
```

What to notice

Classes group data and behavior. Look for object construction and method calls.

Try it yourself

Activity 1. Run the unmodified 04-classes example and record the actual output or observable behavior.

Activity 2. Change one input, literal, or configuration value. Predict the result before rerunning.

Activity 3. Explain one error condition or edge case and how you would test it.

Check your understanding

Question 1. Which line or file provides the entry point for this project?

Question 2. Which concept from this chapter would be most useful in a larger application, and why?

Question 3. What evidence would show that your modified example still works?

Instructor note. Assess the learner’s prediction, observed output, and explanation, not merely successful execution.

Lab A | Source-map investigation

Before editing a program, map the relationship between its entry point, local modules, assets, and README. A source map is a human-readable account of which file has which responsibility.

Source reading: 04-classes/milestone4.fab

```
class BankAccount {
    private String owner
    private Float balance

    init(String owner, Float openingBalance) {
        this.owner = owner
        this.balance = openingBalance
    }

    public func deposit(Float amount) {
        this.balance += amount
    }

    public func getBalance() -> Float {
        return this.balance
    }
}

func main() {
    let account = BankAccount("Tushar", 5000.0)
    account.deposit(2500.0)
    print(account.getBalance())
}
```

1. List the project files and identify which ones contain executable Fubby statements.

2. Find the documented working directory and explain why running from a different directory might fail.

3. Draw a dependency arrow from the entry file to each imported file, package, or external service.

4. State one feature of the example that is explicitly documented and one detail you still need to verify.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab B | Reading program behavior

Practice a dry run: trace control flow without executing the program. For a web or concurrent project, trace a single request or task instead of assuming a globally fixed execution order.

Source reading: 04-classes/README.md

````
# Classes and encapsulation

Milestones: M4.

Construct an account and modify private state through public
methods ...

## Run

With Fubby 0.1.0 on PATH, open a terminal in this folder:

```sh
fubby check milestone4.fab
fubby run milestone4.fab
```

## What to expect

7500

## Try it

Change an input value in the example, rerun it, and compare the
resu ...

[All examples](../README.md)
````

1. Identify the first observable operation and the values it depends on.

2. Write a step-by-step trace for a representative successful path.

3. Identify a branch, function call, or external boundary that could change the trace.

4. Run the official example and compare your trace with actual output or behavior.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab D | Edge cases and failure paths

A program that succeeds on its demonstration input is not automatically robust. Deliberately examine missing values, invalid inputs, empty collections, unavailable services, and incompatible environments.

Source reading: 04-classes/README.md

````
# Classes and encapsulation

Milestones: M4.

Construct an account and modify private state through public
methods ...

## Run

With Fubby 0.1.0 on PATH, open a terminal in this folder:

```sh
fubby check milestone4.fab
fubby run milestone4.fab
```

## What to expect

7500

## Try it

Change an input value in the example, rerun it, and compare the
resu ...

[All examples](../README.md)
````

1. List three edge cases relevant to this chapter, ordered by likelihood.

2. Predict the behavior for one invalid input before running it.

3. Where possible, reproduce a safe failure using disposable local data.

4. Explain whether the failure is handled by Fubby code, the interpreter, or the operating environment.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab F | Verification worksheet

Reproducibility requires more than saying that a program works. A useful verification record identifies the exact command, environment, expected behavior, and observed result.

Source reading: 04-classes/README.md

````
# Classes and encapsulation

Milestones: M4.

Construct an account and modify private state through public
methods ...

## Run

With Fubby 0.1.0 on PATH, open a terminal in this folder:

```sh
fubby check milestone4.fab
fubby run milestone4.fab
```

## What to expect

7500

## Try it

Change an input value in the example, rerun it, and compare the
resu ...

[All examples](../README.md)
````

1. Write the exact command from the project README, including its working directory.

2. Record the interpreter version and whether external services or packages were needed.

3. Define one positive test and one negative test with explicit expected observations.

4. If Shady or another tool applies, record its output and distinguish warnings from failures.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Chapter 05 Inheritance

Official project: examples/05-inheritance/

Milestones: M5. Implement an interface, override a method, and call the parent implementation. With Fubby 0.1.0 on PATH, open a terminal in this folder:

Learning objectives

• Describe inheritance and interfaces.

• Compare parent and derived behavior.

• Explain why shared contracts matter.

Run the official example

```
fubby check milestone5.fab
fubby run milestone5.fab
```

Read the program

Excerpt from: 05-inheritance/milestone5.fab

```
interface Describable {
    func describe() -> String
}

class Account {
    protected String owner
    init(String owner) {
        this.owner = owner
    }
    public func describe() -> String {
        return "Account: " + this.owner
    }
}

class SavingsAccount extends Account implements Describable {
    private Int balance
    init(String owner, Int amount) {
```

```
        this.owner = owner
        this.balance = amount
    }
    public override func describe() -> String {
        return super.describe() + " / savings"
    }
    public func getBalance() -> Int {
        return this.balance
```

The complete runnable source remains in the companion example folder. This is a reading excerpt, not a replacement for the full project.

What to notice

Inheritance reuses a common contract while allowing specialized behavior.

Try it yourself

Activity 1. Run the unmodified 05-inheritance example and record the actual output or observable behavior.

Activity 2. Change one input, literal, or configuration value. Predict the result before rerunning.

Activity 3. Explain one error condition or edge case and how you would test it.

Check your understanding

Question 1. Which line or file provides the entry point for this project?

Question 2. Which concept from this chapter would be most useful in a larger application, and why?

Question 3. What evidence would show that your modified example still works?

Instructor note. Assess the learner’s prediction, observed output, and explanation, not merely successful execution.

Lab A | Source-map investigation

Before editing a program, map the relationship between its entry point, local modules, assets, and README. A source map is a human-readable account of which file has which responsibility.

Source reading: 05-inheritance/milestone5.fab

```
interface Describable {
    func describe() -> String
}

class Account {
    protected String owner
    init(String owner) {
        this.owner = owner
    }
    public func describe() -> String {
        return "Account: " + this.owner
    }
}

class SavingsAccount extends Account implements Describable {
    private Int balance
    init(String owner, Int amount) {
        this.owner = owner
        this.balance = amount
    }
    public override func describe() -> String {
        return super.describe() + " / savings"
    }
    public func getBalance() -> Int {
        return this.balance
    }
}
```

1. List the project files and identify which ones contain executable Fubby statements.

2. Find the documented working directory and explain why running from a different directory might fail.

3. Draw a dependency arrow from the entry file to each imported file, package, or external service.

4. State one feature of the example that is explicitly documented and one detail you still need to verify.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab B | Reading program behavior

Practice a dry run: trace control flow without executing the program. For a web or concurrent project, trace a single request or task instead of assuming a globally fixed execution order.

Source reading: 05-inheritance/README.md

````
# Inheritance and interfaces

Milestones: M5.

Implement an interface, override a method, and call the parent
imple ...

## Run

With Fubby 0.1.0 on PATH, open a terminal in this folder:

```sh
fubby check milestone5.fab
fubby run milestone5.fab
```

## What to expect

Account: Fubby / savings, then 250.

## Try it

Change an input value in the example, rerun it, and compare the
resu ...

[All examples](../README.md)
````

1. Identify the first observable operation and the values it depends on.

2. Write a step-by-step trace for a representative successful path.

3. Identify a branch, function call, or external boundary that could change the trace.

4. Run the official example and compare your trace with actual output or behavior.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab D | Edge cases and failure paths

A program that succeeds on its demonstration input is not automatically robust. Deliberately examine missing values, invalid inputs, empty collections, unavailable services, and incompatible environments.

Source reading: 05-inheritance/README.md

````
# Inheritance and interfaces

Milestones: M5.

Implement an interface, override a method, and call the parent
imple ...

## Run

With Fubby 0.1.0 on PATH, open a terminal in this folder:

```sh
fubby check milestone5.fab
fubby run milestone5.fab
```

## What to expect

Account: Fubby / savings, then 250.

## Try it

Change an input value in the example, rerun it, and compare the
resu ...

[All examples](../README.md)
````

1. List three edge cases relevant to this chapter, ordered by likelihood.

2. Predict the behavior for one invalid input before running it.

3. Where possible, reproduce a safe failure using disposable local data.

4. Explain whether the failure is handled by Fubby code, the interpreter, or the operating environment.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab F | Verification worksheet

Reproducibility requires more than saying that a program works. A useful verification record identifies the exact command, environment, expected behavior, and observed result.

Source reading: 05-inheritance/README.md

````
# Inheritance and interfaces

Milestones: M5.

Implement an interface, override a method, and call the parent
imple ...

## Run

With Fubby 0.1.0 on PATH, open a terminal in this folder:

```sh
fubby check milestone5.fab
fubby run milestone5.fab
```

## What to expect

Account: Fubby / savings, then 250.

## Try it

Change an input value in the example, rerun it, and compare the
resu ...

[All examples](../README.md)
````

1. Write the exact command from the project README, including its working directory.

2. Record the interpreter version and whether external services or packages were needed.

3. Define one positive test and one negative test with explicit expected observations.

4. If Shady or another tool applies, record its output and distinguish warnings from failures.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Chapter 06 Collections Imports Errors

Official project: examples/06-collections-imports-errors/

Milestones: M6. Mutate lists and maps, import a local helper, and catch failures. With Fubby 0.1.0 on PATH, open a terminal in this folder:

Learning objectives

• Create lists and maps.

• Import a local module.

• Handle an exception.

Run the official example

```
fubby check milestone6.fab
fubby run milestone6.fab
fubby check milestone6_complete.fab
fubby run milestone6_complete.fab
```

Read the program

Excerpt from: 06-collections-imports-errors/m6_library.fab

```
func greet() -> String { return "module loaded" }
```

What to notice

Collections store multiple values. Imported files and exception handlers organize larger programs.

Try it yourself

Activity 1. Run the unmodified 06-collections-imports-errors example and record the actual output or observable behavior.

Activity 2. Change one input, literal, or configuration value. Predict the result before rerunning.

Activity 3. Explain one error condition or edge case and how you would test it.

Check your understanding

Question 1. Which line or file provides the entry point for this project?

Question 2. Which concept from this chapter would be most useful in a larger application, and why?

Question 3. What evidence would show that your modified example still works?

Instructor note. Assess the learner’s prediction, observed output, and explanation, not merely successful execution.

Lab A | Source-map investigation

Before editing a program, map the relationship between its entry point, local modules, assets, and README. A source map is a human-readable account of which file has which responsibility.

Source reading: 06-collections-imports-errors/m6_library.fab

```
func greet() -> String { return "module loaded" }
```

1. List the project files and identify which ones contain executable Fubby statements.

2. Find the documented working directory and explain why running from a different directory might fail.

3. Draw a dependency arrow from the entry file to each imported file, package, or external service.

4. State one feature of the example that is explicitly documented and one detail you still need to verify.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab B | Reading program behavior

Practice a dry run: trace control flow without executing the program. For a web or concurrent project, trace a single request or task instead of assuming a globally fixed execution order.

Source reading: 06-collections-imports-errors/milestone6.fab

```
func main() {
    let numbers = [10, 20, 30]
    print(numbers[0])
    numbers.push(40)
    print(len(numbers))
    print(numbers[3])
    let total = 0
    for n in numbers {
        total += n
    }
    print(total)
    print(numbers.pop())
    print(numbers.length())
}
```

1. Identify the first observable operation and the values it depends on.

2. Write a step-by-step trace for a representative successful path.

3. Identify a branch, function call, or external boundary that could change the trace.

4. Run the official example and compare your trace with actual output or behavior.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab D | Edge cases and failure paths

A program that succeeds on its demonstration input is not automatically robust. Deliberately examine missing values, invalid inputs, empty collections, unavailable services, and incompatible environments.

Source reading: 06-collections-imports-errors/README.md

````
# Collections, imports, and exceptions

Milestones: M6.

Mutate lists and maps, import a local helper, and catch failures.

## Run

With Fubby 0.1.0 on PATH, open a terminal in this folder:

```sh
fubby check milestone6.fab
fubby run milestone6.fab
fubby check milestone6_complete.fab
fubby run milestone6_complete.fab
```

## What to expect

List total 100; map values and a greeting; caught missing key.

## Try it

Change an input value in the example, rerun it, and compare the
resu ...

[All examples](../README.md)
````

1. List three edge cases relevant to this chapter, ordered by likelihood.

2. Predict the behavior for one invalid input before running it.

3. Where possible, reproduce a safe failure using disposable local data.

4. Explain whether the failure is handled by Fubby code, the interpreter, or the operating environment.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab F | Verification worksheet

Reproducibility requires more than saying that a program works. A useful verification record identifies the exact command, environment, expected behavior, and observed result.

Source reading: 06-collections-imports-errors/README.md

````
# Collections, imports, and exceptions

Milestones: M6.

Mutate lists and maps, import a local helper, and catch failures.

## Run

With Fubby 0.1.0 on PATH, open a terminal in this folder:

```sh
fubby check milestone6.fab
fubby run milestone6.fab
fubby check milestone6_complete.fab
fubby run milestone6_complete.fab
```

## What to expect

List total 100; map values and a greeting; caught missing key.

## Try it

Change an input value in the example, rerun it, and compare the
resu ...

[All examples](../README.md)
````

1. Write the exact command from the project README, including its working directory.

2. Record the interpreter version and whether external services or packages were needed.

3. Define one positive test and one negative test with explicit expected observations.

4. If Shady or another tool applies, record its output and distinguish warnings from failures.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Chapter 07 Standard Library

Official project: examples/07-standard-library/

Milestones: M7. Transform strings, calculate numbers, and write/read a local text file. With Fubby 0.1.0 on PATH, open a terminal in this folder:

Learning objectives

- Use standard library helpers.
- Explore text and numerical operations.
- Treat file and time operations carefully.

Run the official example

```
fubby check milestone7.fab
fubby run milestone7.fab
```

Read the program

Excerpt from: 07-standard-library/milestone7.fab

```
func main() {
    print(str_upper("fubby"))
    print(str_lower("LANGUAGE"))
    print(str_trim("  clean  "))
    print(str_contains("Fubby language", "language"))
    print(str_replace("hello world", "world", "Fubby"))
    let parts = str_split("a,b,c", ",")
    print(parts[1])
    print(math_abs(-12))
    print(math_sqrt(81))
    print(math_pow(2, 5))
    print(math_min(7, 3))
    print(math_max(7, 3))
    file_write("fubby-m7-test.txt", "Milestone 7")
    print(file_exists("fubby-m7-test.txt"))
    print(file_read("fubby-m7-test.txt"))
    print(clock_unix() > 0)
```

```
}
```

What to notice

Standard-library functions reduce repetitive code, but external effects such as files deserve care.

Try it yourself

Activity 1. Run the unmodified 07-standard-library example and record the actual output or observable behavior.

Activity 2. Change one input, literal, or configuration value. Predict the result before rerunning.

Activity 3. Explain one error condition or edge case and how you would test it.

Check your understanding

Question 1. Which line or file provides the entry point for this project?

Question 2. Which concept from this chapter would be most useful in a larger application, and why?

Question 3. What evidence would show that your modified example still works?

Instructor note. Assess the learner’s prediction, observed output, and explanation, not merely successful execution.

Lab A | Source-map investigation

Before editing a program, map the relationship between its entry point, local modules, assets, and README. A source map is a human-readable account of which file has which responsibility.

Source reading: 07-standard-library/milestone7.fab

```
func main() {
    print(str_upper("fubby"))
    print(str_lower("LANGUAGE"))
    print(str_trim("  clean  "))
    print(str_contains("Fubby language", "language"))
    print(str_replace("hello world", "world", "Fubby"))
    let parts = str_split("a,b,c", ",")
    print(parts[1])
    print(math_abs(-12))
    print(math_sqrt(81))
    print(math_pow(2, 5))
    print(math_min(7, 3))
    print(math_max(7, 3))
    file_write("fubby-m7-test.txt", "Milestone 7")
    print(file_exists("fubby-m7-test.txt"))
    print(file_read("fubby-m7-test.txt"))
    print(clock_unix() > 0)
}
```

1. List the project files and identify which ones contain executable Fubby statements.

2. Find the documented working directory and explain why running from a different directory might fail.

3. Draw a dependency arrow from the entry file to each imported file, package, or external service.

4. State one feature of the example that is explicitly documented and one detail you still need to verify.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab B | Reading program behavior

Practice a dry run: trace control flow without executing the program. For a web or concurrent project, trace a single request or task instead of assuming a globally fixed execution order.

Source reading: 07-standard-library/README.md

````
# Strings, math, files, and time

Milestones: M7.

Transform strings, calculate numbers, and write/read a local text
fi ...

## Run

With Fubby 0.1.0 on PATH, open a terminal in this folder:

```sh
fubby check milestone7.fab
fubby run milestone7.fab
```

## What to expect

FUBBY and other transformed values; file contents Milestone 7.
Creat ...

## Try it

Change an input value in the example, rerun it, and compare the
resu ...

[All examples](../README.md)
````

1. Identify the first observable operation and the values it depends on.

2. Write a step-by-step trace for a representative successful path.

3. Identify a branch, function call, or external boundary that could change the trace.

4. Run the official example and compare your trace with actual output or behavior.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab D | Edge cases and failure paths

A program that succeeds on its demonstration input is not automatically robust. Deliberately examine missing values, invalid inputs, empty collections, unavailable services, and incompatible environments.

Source reading: 07-standard-library/README.md

````
# Strings, math, files, and time

Milestones: M7.

Transform strings, calculate numbers, and write/read a local text
fi ...

## Run

With Fubby 0.1.0 on PATH, open a terminal in this folder:

```sh
fubby check milestone7.fab
fubby run milestone7.fab
```

## What to expect

FUBBY and other transformed values; file contents Milestone 7.
Creat ...

## Try it

Change an input value in the example, rerun it, and compare the
resu ...

[All examples](../README.md)
````

1. List three edge cases relevant to this chapter, ordered by likelihood.

2. Predict the behavior for one invalid input before running it.

3. Where possible, reproduce a safe failure using disposable local data.

4. Explain whether the failure is handled by Fubby code, the interpreter, or the operating environment.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab F | Verification worksheet

Reproducibility requires more than saying that a program works. A useful verification record identifies the exact command, environment, expected behavior, and observed result.

Source reading: 07-standard-library/README.md

````
# Strings, math, files, and time

Milestones: M7.

Transform strings, calculate numbers, and write/read a local text
fi ...

## Run

With Fubby 0.1.0 on PATH, open a terminal in this folder:

```sh
fubby check milestone7.fab
fubby run milestone7.fab
```

## What to expect

FUBBY and other transformed values; file contents Milestone 7.
Creat ...

## Try it

Change an input value in the example, rerun it, and compare the
resu ...

[All examples](../README.md)
````

1. Write the exact command from the project README, including its working directory.

2. Record the interpreter version and whether external services or packages were needed.

3. Define one positive test and one negative test with explicit expected observations.

4. If Shady or another tool applies, record its output and distinguish warnings from failures.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Chapter 08 Runtime

Official project: examples/08-runtime/

Milestones: M9–10, M18. Accumulate a range using compound assignment. These milestones also hardened the runtime rather than adding a separate application API. With Fubby 0.1.0 on PATH, open a terminal in this folder:

Learning objectives

- Describe interpreter runtime constraints.
- Recognize guarded failures.
- Explain why runtime limits are not sandboxing.

Run the official example

```
fubby check milestone10.fab
fubby run milestone10.fab
```

Read the program

Excerpt from: 08-runtime/milestone10.fab

```
func main() {
    let n = 0
    for i in 1..1000 {
        n += i
    }
    print(n)
    print(str_upper("runtime ok"))
}
```

What to notice

An interpreter enforces some runtime limits, but these are not a security boundary.

Try it yourself

Activity 1. Run the unmodified 08-runtime example and record the actual output or observable behavior.

Activity 2. Change one input, literal, or configuration value. Predict the result before rerunning.

Activity 3. Explain one error condition or edge case and how you would test it.

Check your understanding

Question 1. Which line or file provides the entry point for this project?

Question 2. Which concept from this chapter would be most useful in a larger application, and why?

Question 3. What evidence would show that your modified example still works?

Instructor note. Assess the learner’s prediction, observed output, and explanation, not merely successful execution.

Lab A | Source-map investigation

Before editing a program, map the relationship between its entry point, local modules, assets, and README. A source map is a human-readable account of which file has which responsibility.

Source reading: 08-runtime/milestone10.fab

```
func main() {
    let n = 0
    for i in 1..1000 {
        n += i
    }
    print(n)
    print(str_upper("runtime ok"))
}
```

1. List the project files and identify which ones contain executable Fubby statements.

2. Find the documented working directory and explain why running from a different directory might fail.

3. Draw a dependency arrow from the entry file to each imported file, package, or external service.

4. State one feature of the example that is explicitly documented and one detail you still need to verify.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab B | Reading program behavior

Practice a dry run: trace control flow without executing the program. For a web or concurrent project, trace a single request or task instead of assuming a globally fixed execution order.

Source reading: 08-runtime/README.md

````
# Runtime execution

Milestones: M9–10, M18.

Accumulate a range using compound assignment. These milestones also
 ...

## Run

With Fubby 0.1.0 on PATH, open a terminal in this folder:

```sh
fubby check milestone10.fab
fubby run milestone10.fab
```

## What to expect

500500 and RUNTIME OK.

## Try it

Change an input value in the example, rerun it, and compare the
resu ...

[All examples](../README.md)
````

1. Identify the first observable operation and the values it depends on.

2. Write a step-by-step trace for a representative successful path.

3. Identify a branch, function call, or external boundary that could change the trace.

4. Run the official example and compare your trace with actual output or behavior.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab D | Edge cases and failure paths

A program that succeeds on its demonstration input is not automatically robust. Deliberately examine missing values, invalid inputs, empty collections, unavailable services, and incompatible environments.

Source reading: 08-runtime/README.md

````
# Runtime execution

Milestones: M9–10, M18.

Accumulate a range using compound assignment. These milestones also
 ...

## Run

With Fubby 0.1.0 on PATH, open a terminal in this folder:

```sh
fubby check milestone10.fab
fubby run milestone10.fab
```

## What to expect

500500 and RUNTIME OK.

## Try it

Change an input value in the example, rerun it, and compare the
resu ...

[All examples](../README.md)
````

1. List three edge cases relevant to this chapter, ordered by likelihood.

2. Predict the behavior for one invalid input before running it.

3. Where possible, reproduce a safe failure using disposable local data.

4. Explain whether the failure is handled by Fubby code, the interpreter, or the operating environment.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab F | Verification worksheet

Reproducibility requires more than saying that a program works. A useful verification record identifies the exact command, environment, expected behavior, and observed result.

Source reading: 08-runtime/README.md

````
# Runtime execution

Milestones: M9–10, M18.

Accumulate a range using compound assignment. These milestones also
 ...

## Run

With Fubby 0.1.0 on PATH, open a terminal in this folder:

```sh
fubby check milestone10.fab
fubby run milestone10.fab
```

## What to expect

500500 and RUNTIME OK.

## Try it

Change an input value in the example, rerun it, and compare the
resu ...

[All examples](../README.md)
````

1. Write the exact command from the project README, including its working directory.

2. Record the interpreter version and whether external services or packages were needed.

3. Define one positive test and one negative test with explicit expected observations.

4. If Shady or another tool applies, record its output and distinguish warnings from failures.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Chapter 09 Core Library

Official project: examples/09-core-library/

Milestones: M11. Explore Unicode strings, list helpers, maps, JSON, regex, encoding, UTC dates, random values, and environment information. With Fubby 0.1.0 on PATH, open a terminal in this folder:

Learning objectives

• Use Core collection and JSON utilities.

• Understand encoding and environment access.

• Distinguish data formats from application logic.

Run the official example

```
fubby check milestone11.fab
fubby run milestone11.fab
```

Read the program

Excerpt from: 09-core-library/milestone11.fab

```
func main() {
    print(str_length("Fubby □"))
    print(str_substring("Fubby", 1, 4))
    print(str_repeat("ha", 3))
    print(math_sin(0))

    let items = [3, 1, 2]
    print(list_contains(items, 1))
    print(list_slice(items, 1, 3))
    print(list_concat(items, [4]))
    print(list_insert(items, 1, 9))
    print(list_remove(items, 0))

    let data = {"name": "Fubby", "active": true, "count": 3}
    print(map_keys(data))
```

```
print(map_values(data))
let encoded = json_encode(data)
print(encoded)
print(json_encode(json_decode(encoded)))

print(regex_is_match("abc123", "^a.*3$"))
print(encoding_hex_encode("ok"))
print(encoding_hex_decode("6f6b"))
print(encoding_base64_encode("Fubby"))
print(encoding_base64_decode("RnViYnk="))
```

The complete runnable source remains in the companion example folder. This is a reading excerpt, not a replacement for the full project.

What to notice

JSON and encoding helpers make data exchange possible; inspect types and conversion results.

Try it yourself

Activity 1. Run the unmodified 09-core-library example and record the actual output or observable behavior.

Activity 2. Change one input, literal, or configuration value. Predict the result before rerunning.

Activity 3. Explain one error condition or edge case and how you would test it.

Check your understanding

Question 1. Which line or file provides the entry point for this project?

Question 2. Which concept from this chapter would be most useful in a larger application, and why?

Question 3. What evidence would show that your modified example still works?

Instructor note. Assess the learner’s prediction, observed output, and explanation, not merely successful execution.

Lab A | Source-map investigation

Before editing a program, map the relationship between its entry point, local modules, assets, and README. A source map is a human-readable account of which file has which responsibility.

Source reading: 09-core-library/milestone11.fab

```
func main() {
    print(str_length("Fubby □"))
    print(str_substring("Fubby", 1, 4))
    print(str_repeat("ha", 3))
    print(math_sin(0))

    let items = [3, 1, 2]
    print(list_contains(items, 1))
    print(list_slice(items, 1, 3))
    print(list_concat(items, [4]))
    print(list_insert(items, 1, 9))
    print(list_remove(items, 0))

    let data = {"name": "Fubby", "active": true, "count": 3}
    print(map_keys(data))
    print(map_values(data))
    let encoded = json_encode(data)
    print(encoded)
    print(json_encode(json_decode(encoded)))

    print(regex_is_match("abc123", "^a.*3$"))
    print(encoding_hex_encode("ok"))
    print(encoding_hex_decode("6f6b"))
    print(encoding_base64_encode("Fubby"))
    print(encoding_base64_decode("RnViYnk="))
    print(time_format_utc(time_parse_utc("2000-02-29T23:59:59Z")))
    print(time_is_leap_year(2000))
    print(time_days_in_month(2024, 2))
```

1. List the project files and identify which ones contain executable Fubby statements.

2. Find the documented working directory and explain why running from a different directory might fail.

3. Draw a dependency arrow from the entry file to each imported file, package, or external service.

4. State one feature of the example that is explicitly documented and one detail you still need to verify.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab B | Reading program behavior

Practice a dry run: trace control flow without executing the program. For a web or concurrent project, trace a single request or task instead of assuming a globally fixed execution order.

Source reading: 09-core-library/README.md

````
# Collections, JSON, encoding, and environment

Milestones: M11.

Explore Unicode strings, list helpers, maps, JSON, regex, encoding,
 ...

## Run

With Fubby 0.1.0 on PATH, open a terminal in this folder:

```sh
fubby check milestone11.fab
fubby run milestone11.fab
```

## What to expect

JSON round trips, decoded text, leap-year checks, and true for the
c ...

## Try it

Change an input value in the example, rerun it, and compare the
resu ...

[All examples](../README.md)
````

1. Identify the first observable operation and the values it depends on.

2. Write a step-by-step trace for a representative successful path.

3. Identify a branch, function call, or external boundary that could change the trace.

4. Run the official example and compare your trace with actual output or behavior.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab D | Edge cases and failure paths

A program that succeeds on its demonstration input is not automatically robust. Deliberately examine missing values, invalid inputs, empty collections, unavailable services, and incompatible environments.

Source reading: 09-core-library/README.md

````
# Collections, JSON, encoding, and environment

Milestones: M11.

Explore Unicode strings, list helpers, maps, JSON, regex, encoding,
 ...

## Run

With Fubby 0.1.0 on PATH, open a terminal in this folder:

```sh
fubby check milestone11.fab
fubby run milestone11.fab
```

## What to expect

JSON round trips, decoded text, leap-year checks, and true for the
c ...

## Try it

Change an input value in the example, rerun it, and compare the
resu ...

[All examples](../README.md)
````

1. List three edge cases relevant to this chapter, ordered by likelihood.

2. Predict the behavior for one invalid input before running it.

3. Where possible, reproduce a safe failure using disposable local data.

4. Explain whether the failure is handled by Fubby code, the interpreter, or the operating environment.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab F | Verification worksheet

Reproducibility requires more than saying that a program works. A useful verification record identifies the exact command, environment, expected behavior, and observed result.

Source reading: 09-core-library/README.md

````
# Collections, JSON, encoding, and environment

Milestones: M11.

Explore Unicode strings, list helpers, maps, JSON, regex, encoding,
 ...

## Run

With Fubby 0.1.0 on PATH, open a terminal in this folder:

```sh
fubby check milestone11.fab
fubby run milestone11.fab
```

## What to expect

JSON round trips, decoded text, leap-year checks, and true for the
c ...

## Try it

Change an input value in the example, rerun it, and compare the
resu ...

[All examples](../README.md)
````

1. Write the exact command from the project README, including its working directory.

2. Record the interpreter version and whether external services or packages were needed.

3. Define one positive test and one negative test with explicit expected observations.

4. If Shady or another tool applies, record its output and distinguish warnings from failures.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Chapter 10 Moon

Official project: examples/10-moon/

Milestones: M12. Create an in-memory table, bind SQL parameters, query a row, and catch a denied HTTP request. With Fubby 0.1.0 on PATH, open a terminal in this folder:

Learning objectives

- Introduce Moon HTTP and SQLite.
- Use local-only network experiments.
- Explain persistence and network safety.

Run the official example

```
unset FUBBY_NET_ALLOWLIST
fubby check moon_m12.fab
fubby run moon_m12.fab
```

Read the program

Excerpt from: 10-moon/moon_m12.fab

```
func main() {
    let limits = moon.security.limits()
    print(limits["http_timeout_default_ms"])
    print(moon.security.network_allowed("example.invalid"))

    let db = moon.db.sqlite.open(":memory:", {})
    db.execute("CREATE TABLE people (id INTEGER PRIMARY KEY, name
TE ...
    db.execute("INSERT INTO people (name) VALUES (?1)", ["Ada"])
    let rows = db.query("SELECT id, name FROM people WHERE name =
?1 ...
    print(rows[0]["name"])

    try {
        moon.http.get("https://example.invalid/", {})
    } catch (err) {
```

```
            print(str_contains(err, "network access denied"))
        }
    }
```

What to notice

SQLite persists structured data. HTTP requests should be constrained to trusted local test endpoints.

Try it yourself

Activity 1. Run the unmodified 10-moon example and record the actual output or observable behavior.

Activity 2. Change one input, literal, or configuration value. Predict the result before rerunning.

Activity 3. Explain one error condition or edge case and how you would test it.

Check your understanding

Question 1. Which line or file provides the entry point for this project?

Question 2. Which concept from this chapter would be most useful in a larger application, and why?

Question 3. What evidence would show that your modified example still works?

Instructor note. Assess the learner’s prediction, observed output, and explanation, not merely successful execution.

Lab A | Source-map investigation

Before editing a program, map the relationship between its entry point, local modules, assets, and README. A source map is a human-readable account of which file has which responsibility.

Source reading: 10-moon/moon_m12.fab

```
func main() {
    let limits = moon.security.limits()
    print(limits["http_timeout_default_ms"])
    print(moon.security.network_allowed("example.invalid"))

    let db = moon.db.sqlite.open(":memory:", {})
    db.execute("CREATE TABLE people (id INTEGER PRIMARY KEY, name
TE ...
    db.execute("INSERT INTO people (name) VALUES (?1)", ["Ada"])
    let rows = db.query("SELECT id, name FROM people WHERE name =
?1 ...
    print(rows[0]["name"])

    try {
        moon.http.get("https://example.invalid/", {})
    } catch (err) {
        print(str_contains(err, "network access denied"))
    }
}
```

1. List the project files and identify which ones contain executable Fubby statements.

2. Find the documented working directory and explain why running from a different directory might fail.

3. Draw a dependency arrow from the entry file to each imported file, package, or external service.

4. State one feature of the example that is explicitly documented and one detail you still need to verify.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab B | Reading program behavior

Practice a dry run: trace control flow without executing the program. For a web or concurrent project, trace a single request or task instead of assuming a globally fixed execution order.

Source reading: 10-moon/README.md

````
# Moon SQLite and network policy

Milestones: M12.

Create an in-memory table, bind SQL parameters, query a row, and
cat ...

## Run

With Fubby 0.1.0 on PATH, open a terminal in this folder:

```sh
unset FUBBY_NET_ALLOWLIST
fubby check moon_m12.fab
fubby run moon_m12.fab
```

## What to expect

Ada and true for the denied-request check. Run with
FUBBY_NET_ALLOWL ...

## Try it

Change an input value in the example, rerun it, and compare the
resu ...

[All examples](../README.md)
````

1. Identify the first observable operation and the values it depends on.

2. Write a step-by-step trace for a representative successful path.

3. Identify a branch, function call, or external boundary that could change the trace.

4. Run the official example and compare your trace with actual output or behavior.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab D | Edge cases and failure paths

A program that succeeds on its demonstration input is not automatically robust. Deliberately examine missing values, invalid inputs, empty collections, unavailable services, and incompatible environments.

Source reading: 10-moon/README.md

````
# Moon SQLite and network policy

Milestones: M12.

Create an in-memory table, bind SQL parameters, query a row, and
cat ...

## Run

With Fubby 0.1.0 on PATH, open a terminal in this folder:

```sh
unset FUBBY_NET_ALLOWLIST
fubby check moon_m12.fab
fubby run moon_m12.fab
```

## What to expect

Ada and true for the denied-request check. Run with
FUBBY_NET_ALLOWL ...

## Try it

Change an input value in the example, rerun it, and compare the
resu ...

[All examples](../README.md)
````

1. List three edge cases relevant to this chapter, ordered by likelihood.

2. Predict the behavior for one invalid input before running it.

3. Where possible, reproduce a safe failure using disposable local data.

4. Explain whether the failure is handled by Fubby code, the interpreter, or the operating environment.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab F | Verification worksheet

Reproducibility requires more than saying that a program works. A useful verification record identifies the exact command, environment, expected behavior, and observed result.

Source reading: 10-moon/README.md

````
# Moon SQLite and network policy

Milestones: M12.

Create an in-memory table, bind SQL parameters, query a row, and
cat ...

## Run

With Fubby 0.1.0 on PATH, open a terminal in this folder:

```sh
unset FUBBY_NET_ALLOWLIST
fubby check moon_m12.fab
fubby run moon_m12.fab
```

## What to expect

Ada and true for the denied-request check. Run with
FUBBY_NET_ALLOWL ...

## Try it

Change an input value in the example, rerun it, and compare the
resu ...

[All examples](../README.md)
````

1. Write the exact command from the project README, including its working directory.

2. Record the interpreter version and whether external services or packages were needed.

3. Define one positive test and one negative test with explicit expected observations.

4. If Shady or another tool applies, record its output and distinguish warnings from failures.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Chapter 11 Shady

Official project: examples/11-shady/

Milestones: M13. Run tests and inspect an ordinary program with the developer tools. With Fubby 0.1.0 on PATH, open a terminal in this folder:

Learning objectives

- Run Shady developer tools.
- Interpret tests and lint feedback.
- Adopt repeatable development checks.

Run the official example

```
fubby check shady.fab
fubby run shady.fab
fubby check shady_test.fab
fubby run shady_test.fab
fubby shady test shady_test.fab
fubby shady fmt --check shady.fab
fubby shady lint shady.fab
fubby shady profile shady.fab
fubby shady docs shady.fab
```

Read the program

Excerpt from: 11-shady/shady.fab

```
/// Compute a small sum using ordinary Fubby execution.
func sum_to(Int limit) -> Int {
    let total = 0
    for value in 1..limit {
        total += value
    }
    return total
}

/// Entry point for Shady debug and profile demonstrations.
```

```
func main() {
    let total = sum_to(6)
    print(total)
}
```

What to notice

Tests catch regressions; formatters and linters improve consistency, not correctness guarantees.

Try it yourself

Activity 1. Run the unmodified 11-shady example and record the actual output or observable behavior.

Activity 2. Change one input, literal, or configuration value. Predict the result before rerunning.

Activity 3. Explain one error condition or edge case and how you would test it.

Check your understanding

Question 1. Which line or file provides the entry point for this project?

Question 2. Which concept from this chapter would be most useful in a larger application, and why?

Question 3. What evidence would show that your modified example still works?

Instructor note. Assess the learner’s prediction, observed output, and explanation, not merely successful execution.

Lab A | Source-map investigation

Before editing a program, map the relationship between its entry point, local modules, assets, and README. A source map is a human-readable account of which file has which responsibility.

Source reading: 11-shady/shady.fab

```
/// Compute a small sum using ordinary Fubby execution.
func sum_to(Int limit) -> Int {
    let total = 0
    for value in 1..limit {
        total += value
    }
    return total
}

/// Entry point for Shady debug and profile demonstrations.
func main() {
    let total = sum_to(6)
    print(total)
}
```

1. List the project files and identify which ones contain executable Fubby statements.

2. Find the documented working directory and explain why running from a different directory might fail.

3. Draw a dependency arrow from the entry file to each imported file, package, or external service.

4. State one feature of the example that is explicitly documented and one detail you still need to verify.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

Lab B | Reading program behavior

Practice a dry run: trace control flow without executing the program. For a web or concurrent project, trace a single request or task instead of assuming a globally fixed execution order.

Source reading: 11-shady/shady_test.fab

```
/// Tests use zero-argument Void functions and throw on failure.
func test_arithmetic() {
    if 2 + 2 != 4 {
        throw "expected 2 + 2 to equal 4"
    }
}

/// Each test receives a fresh interpreter runtime budget.
func test_collections() {
    let values = [1, 2, 3]
    if values.length() != 3 {
        throw "expected three values"
    }
}

func main() {
    test_arithmetic()
    test_collections()
    print("Shady examples passed")
}
```

1. Identify the first observable operation and the values it depends on.

2. Write a step-by-step trace for a representative successful path.

3. Identify a branch, function call, or external boundary that could change the trace.

4. Run the official example and compare your trace with actual output or behavior.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab D | Edge cases and failure paths

A program that succeeds on its demonstration input is not automatically robust. Deliberately examine missing values, invalid inputs, empty collections, unavailable services, and incompatible environments.

Source reading: 11-shady/README.md

````
# Shady developer tools

Milestones: M13.

Run tests and inspect an ordinary program with the developer tools.

## Run

With Fubby 0.1.0 on PATH, open a terminal in this folder:

```sh
fubby check shady.fab
fubby run shady.fab
fubby check shady_test.fab
fubby run shady_test.fab
fubby shady test shady_test.fab
fubby shady fmt --check shady.fab
fubby shady lint shady.fab
fubby shady profile shady.fab
fubby shady docs shady.fab
```

## What to expect

Two passing tests; the sum program prints 21.

## Try it
````

1. List three edge cases relevant to this chapter, ordered by likelihood.

2. Predict the behavior for one invalid input before running it.

3. Where possible, reproduce a safe failure using disposable local data.

4. Explain whether the failure is handled by Fubby code, the interpreter, or the operating environment.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab F | Verification worksheet

Reproducibility requires more than saying that a program works. A useful verification record identifies the exact command, environment, expected behavior, and observed result.

Source reading: 11-shady/README.md

````
# Shady developer tools

Milestones: M13.

Run tests and inspect an ordinary program with the developer tools.

## Run

With Fubby 0.1.0 on PATH, open a terminal in this folder:

```sh
fubby check shady.fab
fubby run shady.fab
fubby check shady_test.fab
fubby run shady_test.fab
fubby shady test shady_test.fab
fubby shady fmt --check shady.fab
fubby shady lint shady.fab
fubby shady profile shady.fab
fubby shady docs shady.fab
```

## What to expect

Two passing tests; the sum program prints 21.

## Try it
````

1. Write the exact command from the project README, including its working directory.

2. Record the interpreter version and whether external services or packages were needed.

3. Define one positive test and one negative test with explicit expected observations.

4. If Shady or another tool applies, record its output and distinguish warnings from failures.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Chapter 12 Snow

Official project: examples/12-snow/

Milestone M14: routes, HTML templates, static CSS/JavaScript, middleware, JSON responses, and parameterized SQLite CRUD. With Fubby 0.1.0 on PATH, run from this folder:

Learning objectives

- Start and stop a Snow web app.
- Identify page, API, and static assets.
- Explain request and response handling.

Run the official example

```
fubby check app.fab
fubby snow serve app.fab --port 8080
```

Read the program

Excerpt from: 12-snow/app.fab

```
/// Snow Notes: a local SQLite-backed web application.
func main() {
    let db = moon.db.sqlite.open("snow-notes.db", {})
    db.execute("CREATE TABLE IF NOT EXISTS notes (id INTEGER
PRIMARY ...
    snow.get("/", "home")
    snow.get("/about", "about")
    snow.get("/hello/:name", "hello")
    snow.get("/api/notes", "list_notes")
    snow.post("/api/notes", "create_note")
    snow.get("/api/notes/:id", "read_note")
    snow.put("/api/notes/:id", "update_note")
    snow.patch("/api/notes/:id", "update_note")
    snow.delete("/api/notes/:id", "delete_note")
    snow.static("/assets", "assets")
    snow.before("request_context")
    snow.after("response_headers")
```

```
    snow.on_error("application_error")
}

func request_context() {
    let request = snow.request()
    request["state"].set("framework", "Snow")
}

func response_headers() {
```

The complete runnable source remains in the companion example folder. This is a reading excerpt, not a replacement for the full project.

What to notice

Web applications combine routes, responses, and static assets. Stop the server after experimenting.

Try it yourself

Activity 1. Run the unmodified 12-snow example and record the actual output or observable behavior.

Activity 2. Change one input, literal, or configuration value. Predict the result before rerunning.

Activity 3. Explain one error condition or edge case and how you would test it.

Check your understanding

Question 1. Which line or file provides the entry point for this project?

Question 2. Which concept from this chapter would be most useful in a larger application, and why?

Question 3. What evidence would show that your modified example still works?

Instructor note. Assess the learner’s prediction, observed output, and explanation, not merely successful execution.

Lab A | Source-map investigation

Before editing a program, map the relationship between its entry point, local modules, assets, and README. A source map is a human-readable account of which file has which responsibility.

Source reading: 12-snow/app.fab

```
/// Snow Notes: a local SQLite-backed web application.
func main() {
    let db = moon.db.sqlite.open("snow-notes.db", {})
    db.execute("CREATE TABLE IF NOT EXISTS notes (id INTEGER
PRIMARY ...
    snow.get("/", "home")
    snow.get("/about", "about")
    snow.get("/hello/:name", "hello")
    snow.get("/api/notes", "list_notes")
    snow.post("/api/notes", "create_note")
    snow.get("/api/notes/:id", "read_note")
    snow.put("/api/notes/:id", "update_note")
    snow.patch("/api/notes/:id", "update_note")
    snow.delete("/api/notes/:id", "delete_note")
    snow.static("/assets", "assets")
    snow.before("request_context")
    snow.after("response_headers")
    snow.on_error("application_error")
}

func request_context() {
    let request = snow.request()
    request["state"].set("framework", "Snow")
}

func response_headers() {
    snow.header("X-Framework", snow.request()["state"]["framework"])
}
```

1. List the project files and identify which ones contain executable Fubby statements.

2. Find the documented working directory and explain why running from a different directory might fail.

3. Draw a dependency arrow from the entry file to each imported file, package, or external service.

4. State one feature of the example that is explicitly documented and one detail you still need to verify.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab B | Reading program behavior

Practice a dry run: trace control flow without executing the program. For a web or concurrent project, trace a single request or task instead of assuming a globally fixed execution order.

Source reading: 12-snow/README.md

````
# Snow Notes

Milestone M14: routes, HTML templates, static CSS/JavaScript,
middle ...

With Fubby 0.1.0 on PATH, run from this folder:

```sh
fubby check app.fab
fubby snow serve app.fab --port 8080
```

Open http://127.0.0.1:8080/. Add a note using the form. Reload to
se ...

From a second terminal, exercise the API:

```sh
curl -X POST http://127.0.0.1:8080/api/notes -H 'Content-Type:
appli ...
curl http://127.0.0.1:8080/api/notes
```

The POST returns a created note with an ID; GET returns the notes
ar ...

Creates `snow-notes.db` in the working directory. Keep the `assets`
 ...

[All examples](../README.md)
````

1. Identify the first observable operation and the values it depends on.

2. Write a step-by-step trace for a representative successful path.

3. Identify a branch, function call, or external boundary that could change the trace.

4. Run the official example and compare your trace with actual output or behavior.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab D | Edge cases and failure paths

A program that succeeds on its demonstration input is not automatically robust. Deliberately examine missing values, invalid inputs, empty collections, unavailable services, and incompatible environments.

Source reading: 12-snow/README.md

````
# Snow Notes

Milestone M14: routes, HTML templates, static CSS/JavaScript,
middle ...

With Fubby 0.1.0 on PATH, run from this folder:

```sh
fubby check app.fab
fubby snow serve app.fab --port 8080
```

Open http://127.0.0.1:8080/. Add a note using the form. Reload to
se ...

From a second terminal, exercise the API:

```sh
curl -X POST http://127.0.0.1:8080/api/notes -H 'Content-Type:
appli ...
curl http://127.0.0.1:8080/api/notes
```

The POST returns a created note with an ID; GET returns the notes
ar ...

Creates `snow-notes.db` in the working directory. Keep the `assets`
 ...

[All examples](../README.md)
````

1. List three edge cases relevant to this chapter, ordered by likelihood.

2. Predict the behavior for one invalid input before running it.

3. Where possible, reproduce a safe failure using disposable local data.

4. Explain whether the failure is handled by Fubby code, the interpreter, or the operating environment.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab F | Verification worksheet

Reproducibility requires more than saying that a program works. A useful verification record identifies the exact command, environment, expected behavior, and observed result.

Source reading: 12-snow/README.md

````
# Snow Notes

Milestone M14: routes, HTML templates, static CSS/JavaScript,
middle ...

With Fubby 0.1.0 on PATH, run from this folder:

```sh
fubby check app.fab
fubby snow serve app.fab --port 8080
```

Open http://127.0.0.1:8080/. Add a note using the form. Reload to
se ...

From a second terminal, exercise the API:

```sh
curl -X POST http://127.0.0.1:8080/api/notes -H 'Content-Type:
appli ...
curl http://127.0.0.1:8080/api/notes
```

The POST returns a created note with an ID; GET returns the notes
ar ...

Creates `snow-notes.db` in the working directory. Keep the `assets`
 ...

[All examples](../README.md)
````

1. Write the exact command from the project README, including its working directory.

2. Record the interpreter version and whether external services or packages were needed.

3. Define one positive test and one negative test with explicit expected observations.

4. If Shady or another tool applies, record its output and distinguish warnings from failures.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Chapter 13 Bela

Official project: examples/13-bela/

Milestones: M15. Spawn a task with input, join its result, and send a message through a bounded channel. With Fubby 0.1.0 on PATH, open a terminal in this folder:

Learning objectives

- Explore Bela tasks and channels.
- Distinguish concurrency from parallelism.
- Reason about ordering and communication.

Run the official example

```
fubby check bela.fab
fubby run bela.fab
```

Read the program

Excerpt from: 13-bela/bela.fab

```
func calculate() {
    let input = bela.input()
    bela.result(input["value"] * input["value"])
}

func main() {
    let task = bela.spawn("calculate", {"value": 12})
    print(bela.status(task))
    print(bela.join(task))

    let channel = bela.channel(2)
    bela.send(channel, "ready")
    print(bela.recv(channel))
    bela.close(channel)
}
```

What to notice

Concurrent tasks can interleave. Communication through channels should be understood explicitly.

Try it yourself

Activity 1. Run the unmodified 13-bela example and record the actual output or observable behavior.

Activity 2. Change one input, literal, or configuration value. Predict the result before rerunning.

Activity 3. Explain one error condition or edge case and how you would test it.

Check your understanding

Question 1. Which line or file provides the entry point for this project?

Question 2. Which concept from this chapter would be most useful in a larger application, and why?

Question 3. What evidence would show that your modified example still works?

Instructor note. Assess the learner’s prediction, observed output, and explanation, not merely successful execution.

Lab A | Source-map investigation

Before editing a program, map the relationship between its entry point, local modules, assets, and README. A source map is a human-readable account of which file has which responsibility.

Source reading: 13-bela/bela.fab

```
func calculate() {
    let input = bela.input()
    bela.result(input["value"] * input["value"])
}

func main() {
    let task = bela.spawn("calculate", {"value": 12})
    print(bela.status(task))
    print(bela.join(task))

    let channel = bela.channel(2)
    bela.send(channel, "ready")
    print(bela.recv(channel))
    bela.close(channel)
}
```

1. List the project files and identify which ones contain executable Fubby statements.

2. Find the documented working directory and explain why running from a different directory might fail.

3. Draw a dependency arrow from the entry file to each imported file, package, or external service.

4. State one feature of the example that is explicitly documented and one detail you still need to verify.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

Lab B | Reading program behavior

Practice a dry run: trace control flow without executing the program. For a web or concurrent project, trace a single request or task instead of assuming a globally fixed execution order.

Source reading: 13-bela/README.md

````
# Bela tasks and channels

Milestones: M15.

Spawn a task with input, join its result, and send a message
through ...

## Run

With Fubby 0.1.0 on PATH, open a terminal in this folder:

```sh
fubby check bela.fab
fubby run bela.fab
```

## What to expect

A task status (timing-dependent), 144, and ready.

## Try it

Change an input value in the example, rerun it, and compare the
resu ...

[All examples](../README.md)
````

1. Identify the first observable operation and the values it depends on.

2. Write a step-by-step trace for a representative successful path.

3. Identify a branch, function call, or external boundary that could change the trace.

4. Run the official example and compare your trace with actual output or behavior.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab D | Edge cases and failure paths

A program that succeeds on its demonstration input is not automatically robust. Deliberately examine missing values, invalid inputs, empty collections, unavailable services, and incompatible environments.

Source reading: 13-bela/README.md

````
# Bela tasks and channels

Milestones: M15.

Spawn a task with input, join its result, and send a message
through ...

## Run

With Fubby 0.1.0 on PATH, open a terminal in this folder:

```sh
fubby check bela.fab
fubby run bela.fab
```

## What to expect

A task status (timing-dependent), 144, and ready.

## Try it

Change an input value in the example, rerun it, and compare the
resu ...

[All examples](../README.md)
````

1. List three edge cases relevant to this chapter, ordered by likelihood.

2. Predict the behavior for one invalid input before running it.

3. Where possible, reproduce a safe failure using disposable local data.

4. Explain whether the failure is handled by Fubby code, the interpreter, or the operating environment.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab F | Verification worksheet

Reproducibility requires more than saying that a program works. A useful verification record identifies the exact command, environment, expected behavior, and observed result.

Source reading: 13-bela/README.md

````
# Bela tasks and channels

Milestones: M15.

Spawn a task with input, join its result, and send a message
through ...

## Run

With Fubby 0.1.0 on PATH, open a terminal in this folder:

```sh
fubby check bela.fab
fubby run bela.fab
```

## What to expect

A task status (timing-dependent), 144, and ready.

## Try it

Change an input value in the example, rerun it, and compare the
resu ...

[All examples](../README.md)
````

1. Write the exact command from the project README, including its working directory.

2. Record the interpreter version and whether external services or packages were needed.

3. Define one positive test and one negative test with explicit expected observations.

4. If Shady or another tool applies, record its output and distinguish warnings from failures.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Chapter 14 Smokey

Official project: examples/14-smokey/

This example demonstrates an application depending on `greeting`, which in turn depends on `text-util`. The registry is a local directory; these commands do not use the network. With Fubby 0.1.0 on `PATH`, start in this example folder.

Learning objectives

- Describe Smokey packages.
- Inspect package metadata and dependencies.
- Validate a local package workflow.

Run the official example

```
REGISTRY="$(mktemp -d "${TMPDIR:-/tmp}/fubby-smokey.XXXXXX")"

cd text-util
fubby pkg validate
fubby pkg publish --registry "$REGISTRY"

cd ../greeting
fubby pkg publish --registry "$REGISTRY"

cd ../app
fubby pkg add greeting '^1.0.0'
fubby pkg validate --registry "$REGISTRY"
fubby pkg install --registry "$REGISTRY"
fubby pkg list
fubby run src/main.fab
```

Read the program

Excerpt from: 14-smokey/app/src/main.fab

```
import "package:greeting/src/lib.fab"

func main() {
```

```
    print(greeting())
}
```

What to notice

Package metadata names the software and its dependencies. Prefer local packages while learning.

Try it yourself

Activity 1. Run the unmodified 14-smokey example and record the actual output or observable behavior.

Activity 2. Change one input, literal, or configuration value. Predict the result before rerunning.

Activity 3. Explain one error condition or edge case and how you would test it.

Check your understanding

Question 1. Which line or file provides the entry point for this project?

Question 2. Which concept from this chapter would be most useful in a larger application, and why?

Question 3. What evidence would show that your modified example still works?

Instructor note. Assess the learner’s prediction, observed output, and explanation, not merely successful execution.

Lab A | Source-map investigation

Before editing a program, map the relationship between its entry point, local modules, assets, and README. A source map is a human-readable account of which file has which responsibility.

Source reading: 14-smokey/app/src/main.fab

```
import "package:greeting/src/lib.fab"

func main() {
    print(greeting())
}
```

1. List the project files and identify which ones contain executable Fubby statements.

2. Find the documented working directory and explain why running from a different directory might fail.

3. Draw a dependency arrow from the entry file to each imported file, package, or external service.

4. State one feature of the example that is explicitly documented and one detail you still need to verify.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab B | Reading program behavior

Practice a dry run: trace control flow without executing the program. For a web or concurrent project, trace a single request or task instead of assuming a globally fixed execution order.

Source reading: 14-smokey/greeting/src/lib.fab

```
import "package:text-util/src/lib.fab"

func greeting() -> String {
    return package_message()
}
```

1. Identify the first observable operation and the values it depends on.

2. Write a step-by-step trace for a representative successful path.

3. Identify a branch, function call, or external boundary that could change the trace.

4. Run the official example and compare your trace with actual output or behavior.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab D | Edge cases and failure paths

A program that succeeds on its demonstration input is not automatically robust. Deliberately examine missing values, invalid inputs, empty collections, unavailable services, and incompatible environments.

Source reading: 14-smokey/README.md

````
# Smokey local registry example

This example demonstrates an application depending on `greeting`,
wh ...

```sh
REGISTRY="$(mktemp -d "${TMPDIR:-/tmp}/fubby-smokey.XXXXXX")"

cd text-util
fubby pkg validate
fubby pkg publish --registry "$REGISTRY"

cd ../greeting
fubby pkg publish --registry "$REGISTRY"

cd ../app
fubby pkg add greeting '^1.0.0'
fubby pkg validate --registry "$REGISTRY"
fubby pkg install --registry "$REGISTRY"
fubby pkg list
fubby run src/main.fab
```

The application imports the greeting package with
`package:greeting/ ...

Expected result: package validation and installation succeed, and
th ...

[All examples](../README.md)
````

1. List three edge cases relevant to this chapter, ordered by likelihood.

2. Predict the behavior for one invalid input before running it.

3. Where possible, reproduce a safe failure using disposable local data.

4. Explain whether the failure is handled by Fubby code, the interpreter, or the operating environment.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab F | Verification worksheet

Reproducibility requires more than saying that a program works. A useful verification record identifies the exact command, environment, expected behavior, and observed result.

Source reading: 14-smokey/greeting/fubby.toml

```
manifest-version = 1

[package]
name = "greeting"
version = "1.0.0"
fubby = ">=0.1.0, <0.2.0"

[dependencies]
text-util = "^1.0.0"
```

1. Write the exact command from the project README, including its working directory.

2. Record the interpreter version and whether external services or packages were needed.

3. Define one positive test and one negative test with explicit expected observations.

4. If Shady or another tool applies, record its output and distinguish warnings from failures.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Chapter 15 Integrated App

Official project: examples/15-integrated-app/

Milestone M17 combines Core helpers, Moon SQLite/HTTP, Snow pages and assets, Bela tasks/channels, Shady tests, and a local Smokey dependency. With Fubby 0.1.0 on PATH, start in this folder:

Learning objectives

- Read an integrated application structure.
- Trace interactions across modules.
- Test a complete local application.

Run the official example

```
cd app
fubby pkg validate
fubby pkg install
fubby shady test tests/preview_tests.fab
fubby snow serve src/main.fab --port 8080 --workers 4
--queue-capaci ...
```

Read the program

Excerpt from: 15-integrated-app/app/src/main.fab

```
import "package:preview-kit/src/lib.fab"

func main() {
    let db = moon.db.sqlite.open("preview.db", {})
    db.execute("CREATE TABLE IF NOT EXISTS notes (id INTEGER
PRIMARY ...
    snow.get("/", "home")
    snow.get("/about", "about")
    snow.get("/runtime", "runtime_page")
    snow.get("/api/notes", "list_notes")
    snow.post("/api/notes", "create_note")
```

```
    snow.get("/api/notes/:id", "read_note")
    snow.patch("/api/notes/:id", "update_note")
    snow.delete("/api/notes/:id", "delete_note")
    snow.get("/api/probe/:token", "request_probe")
    snow.get("/api/upstream", "upstream_probe")
    snow.static("/assets", "../assets")
    snow.before("request_context")
    snow.after("response_headers")
    snow.on_error("application_error")
}

func request_context() {
    let request = snow.request()
    request["state"].set("preview", "Milestone 17")
}
```

The complete runnable source remains in the companion example folder. This is a reading excerpt, not a replacement for the full project.

What to notice

A complete application integrates the language, tools, packages, and services studied earlier.

Try it yourself

Activity 1. Run the unmodified 15-integrated-app example and record the actual output or observable behavior.

Activity 2. Change one input, literal, or configuration value. Predict the result before rerunning.

Activity 3. Explain one error condition or edge case and how you would test it.

Check your understanding

Question 1. Which line or file provides the entry point for this project?

Question 2. Which concept from this chapter would be most useful in a larger application, and why?

Question 3. What evidence would show that your modified example still works?

Instructor note. Assess the learner’s prediction, observed output, and explanation, not merely successful execution.

Lab A | Source-map investigation

Before editing a program, map the relationship between its entry point, local modules, assets, and README. A source map is a human-readable account of which file has which responsibility.

Source reading: 15-integrated-app/app/src/main.fab

```
import "package:preview-kit/src/lib.fab"

func main() {
    let db = moon.db.sqlite.open("preview.db", {})
    db.execute("CREATE TABLE IF NOT EXISTS notes (id INTEGER
PRIMARY ...
    snow.get("/", "home")
    snow.get("/about", "about")
    snow.get("/runtime", "runtime_page")
    snow.get("/api/notes", "list_notes")
    snow.post("/api/notes", "create_note")
    snow.get("/api/notes/:id", "read_note")
    snow.patch("/api/notes/:id", "update_note")
    snow.delete("/api/notes/:id", "delete_note")
    snow.get("/api/probe/:token", "request_probe")
    snow.get("/api/upstream", "upstream_probe")
    snow.static("/assets", "../assets")
    snow.before("request_context")
    snow.after("response_headers")
    snow.on_error("application_error")
}

func request_context() {
    let request = snow.request()
    request["state"].set("preview", "Milestone 17")
}

func response_headers() {
    snow.header("X-Fubby-Preview",
snow.request()["state"]["preview" ...
```

1. List the project files and identify which ones contain executable Fubby statements.

2. Find the documented working directory and explain why running from a different directory might fail.

3. Draw a dependency arrow from the entry file to each imported file, package, or external service.

4. State one feature of the example that is explicitly documented and one detail you still need to verify.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab B | Reading program behavior

Practice a dry run: trace control flow without executing the program. For a web or concurrent project, trace a single request or task instead of assuming a globally fixed execution order.

Source reading: 15-integrated-app/app/src/profile.fab

```
import "package:preview-kit/src/lib.fab"

func main() {
    let value = preview_greeting("  Fubby developer  ")
    print(value)
}
```

1. Identify the first observable operation and the values it depends on.

2. Write a step-by-step trace for a representative successful path.

3. Identify a branch, function call, or external boundary that could change the trace.

4. Run the official example and compare your trace with actual output or behavior.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab D | Edge cases and failure paths

A program that succeeds on its demonstration input is not automatically robust. Deliberately examine missing values, invalid inputs, empty collections, unavailable services, and incompatible environments.

Source reading: 15-integrated-app/packages/preview-kit/src/lib.fab

```
/// Normalize a user supplied title using Fubby Core string helpers.
func preview_title(String title_text) -> String {
    return str_trim(title_text)
}

/// Build a short greeting used by the web application's About page.
func preview_greeting(String person_name) -> String {
    return "Hello, " + str_trim(person_name) + "!"
}
```

1. List three edge cases relevant to this chapter, ordered by likelihood.

2. Predict the behavior for one invalid input before running it.

3. Where possible, reproduce a safe failure using disposable local data.

4. Explain whether the failure is handled by Fubby code, the interpreter, or the operating environment.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Lab F | Verification worksheet

Reproducibility requires more than saying that a program works. A useful verification record identifies the exact command, environment, expected behavior, and observed result.

Source reading: 15-integrated-app/app/fubby.toml

```
manifest-version = 1

[package]
name = "integrated-preview"
version = "0.1.0"
fubby = ">=0.1.0, <0.2.0"

[dependencies]
preview-kit = { version = "^0.1.0", path =
"../packages/preview-kit" ...
```

1. Write the exact command from the project README, including its working directory.

2. Record the interpreter version and whether external services or packages were needed.

3. Define one positive test and one negative test with explicit expected observations.

4. If Shady or another tool applies, record its output and distinguish warnings from failures.

Student evidence: Record the command used, observed result, one explanation, and one question to investigate. If a feature is unsupported by your installed version, document the limitation instead of inventing an output.

Notes / observations: _____

_____

_____

Capstone project Build a small local application

The final official project, examples/15-integrated-app, is a model for combining Fubby packages, modules, application logic, and local services. Use it as a reference, not as a template to copy blindly.

Project brief

Build a small local study tracker, task log, or reading journal. It should accept a few entries, store or organize them, report useful summaries, and include automated checks. Use only features supported by your installed Fubby build.

Required deliverables

- A README explaining the project and exact run command.
- At least two meaningful functions and one collection.
- An input validation or error-handling path.
- A repeatable Shady test or documented manual test.
- A short reflection describing one bug and how it was resolved.

Assessment rubric

- Correct behavior and reproducible instructions: 40%.
- Clear decomposition and readable code: 25%.
- Testing and edge cases: 20%.
- Explanation and reflection: 15%.

Extension challenges

Optional extensions include a class-based model, a Smokey local package, a Moon SQLite store, a Snow local web interface, or Bela tasks. Only add a feature after the simpler version works.

Glossary

API. An interface that lets software components exchange requests and responses.

Argument. A value supplied to a function or command.

Bela. Fubby subsystem demonstrated with tasks and channels.

Class. A definition grouping object state and behavior.

CLI. Command-line interface used to invoke Fubby tools.

Collection. A data structure such as a list or map.

Exception. A runtime condition that can be thrown and handled.

Fubby Core. Library features for common data operations.

Function. A named reusable block of instructions.

Interpreter. A program that executes source code without requiring a conventional compiled application build.

JSON. A structured text data format commonly used for interchange.

Module. A source unit that can be imported by another program.

Moon. Fubby HTTP and SQLite subsystem.

Package. A distributable unit with metadata and source files.

REPL. Read-evaluate-print loop; Fubby preview input is stateless across lines.

Shady. Fubby development tooling, including testing and code checks.

Smokey. Fubby package-management subsystem.

Snow. Fubby web framework for local pages and APIs.

Source file. A text file containing Fubby code, conventionally ending in .fab.

Variable. A name associated with a value.

Troubleshooting and responsible use

Command not found

Check the install prefix and PATH; use the actual executable path if needed.

Wrong archive layout

Inspect the M20 tarball before extracting. Older README instructions may refer to a different preview package.

Example cannot locate a file

Run from the working directory specified by the example README; check relative imports.

Port already in use

Stop a prior local Snow or Moon server before launching another on the same port.

Unexpected runtime error

Run fubby check where supported, inspect the exact error, and reduce to a small reproducible example.

Checksum mismatch

Do not install the archive. Obtain a fresh official download and compare the published checksum.

Security

Do not run unknown .fab code, expose development servers to the public internet, or use preview features for sensitive data.

Index

API 118, 146
Bela 129, 146
classes 39, 49
CLI 10, 108
collections 60, 87
concurrency 129
control flow 19
Core library 87
exceptions 60
functions 29
inheritance 49
installation 10
integrated application 146, 154
JSON 87
Moon 98
packages 139, 146
recursion 29
REPL 10
runtime 78
Shady 108, 146
Smokey 139, 146
Snow 118, 146
SQLite 98
standard library 68
testing 108, 154
variables 11, 19
web framework 118